

Protocolo de Comunicación UART

STM32H723 ↔ ESP32

Coheteros

7 de septiembre de 2026

Todos los datos son **little-endian**, estructuras empaquetadas, sin relleno. El header 0xCAFE se transmite LSB primero: 0xFE, 0xCA. Footer siempre 0xBE.

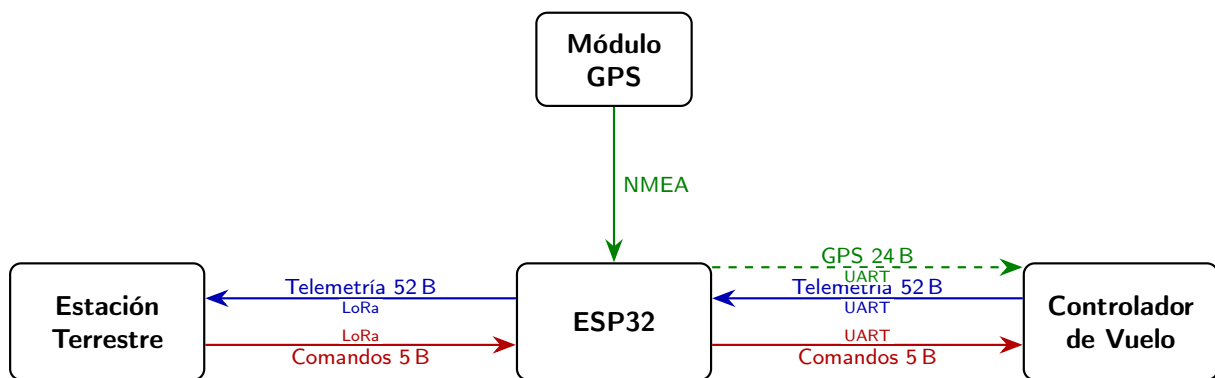


Figura 1: Flujo de datos. Línea punteada = trama construida por ESP32.

1. Telemetría (CV → ESP32 → ET)

Paquete de **52 bytes**. El ESP32 lo reenvía por LoRa **sin modificación**. No tiene bytes CMD ni LEN – solo sync word y footer.

Offset	Bytes	Tipo	Campo	Codificación
0	2	uint16	Sync	0xCAFE
2	4	uint32	Tick	ms (crudo)
6	2	int16	AccelX	m/s ² truncado
8	2	int16	AccelY	m/s ² truncado
10	2	int16	AccelZ	m/s ² truncado
12	2	int16	GyroX	deg/s truncado
14	2	int16	GyroY	deg/s truncado
16	2	int16	GyroZ	deg/s truncado
18	2	int16	PressurePa	Pa ÷ 10
20	1	int8	TemperatureC	°C truncado
21	4	int32	Latitude	deg × 10 ⁷
25	4	int32	Longitude	deg × 10 ⁷
29	4	int32	GPSAltitude	m × 100
33	1	uint8	Satellites	crudo
34	4	int32	BaroAltitude	m × 100
38	4	int32	BaroVelocity	m/s × 100

Offset	Bytes	Tipo	Campo	Codificación
42	4	uint32	FaultFlags	máscara de bits
46	2	int16	BatteryVoltage	$V \times 10$
48	1	uint8	State	enum
49	1	uint8	RelayState	máscara de bits
50	1	uint8	LastCommand	enum
51	1	uint8	SyncEnd	0xBE

Cuadro 1: Paquete de telemetría (52 bytes).

1.1. Pseudocódigo: recepción de telemetría en ESP32

Listing 1: ESP32: recibir telemetría y reenviar por LoRa.

```
// Buffer circular UART. Buscar sync word 0xFE 0xCA.
// Cuando se detecta, leer 52 bytes totales (incluyendo sync).
// Verificar que byte[51] == 0xBE (footer).
// Si es valido, enviar los 52 bytes por LoRa sin modificar.
// Si el footer no coincide, descartar y seguir buscando sync.

#define TELEMETRY_SIZE 52

if (uart_byte == 0xFE) {
    buffer[0] = 0xFE;
    uart_read(buffer + 1, TELEMETRY_SIZE - 1);
    if (buffer[1] == 0xCA && buffer[51] == 0xBE) {
        lora_send(buffer, TELEMETRY_SIZE);
    }
}
```

2. Comandos (ET → ESP32 → CV)

El ESP32 reenvía los bytes crudos recibidos por LoRa directamente al UART. **Sin parseo, sin validación, sin reempaquetado.**

Trama de **5 bytes** (comandos del operador no llevan payload):

Offset	Bytes	Tipo	Valor	Descripción
0	1	uint8	0xFE	Sync LSB
1	1	uint8	0xCA	Sync MSB
2	1	uint8	CMD	Tipo de comando
3	1	uint8	0x00	Longitud payload (cero)
4	1	uint8	0xBE	Footer

Cuadro 2: Trama de comando (5 bytes).

Valores válidos de CMD para comandos del operador:

3. Datos GPS (GPS → ESP32 → CV)

Única trama que el ESP32 **construye**. Parsea la sentencia NMEA **GGA** (\$GPGGA o \$GNGGA) del módulo GPS y empaqueta en una trama de comando con tipo 0x20. **24 bytes** totales.

CMD	Acción
0x01	Reset
0x02	Aborto en tierra
0x03	Calibración
0x04	Drogue
0x05	Aterrizado

Cuadro 3: Comandos del operador.

Offset	Bytes	Tipo	Valor	Descripción
0	1	uint8	0xFE	Sync LSB
1	1	uint8	0xCA	Sync MSB
2	1	uint8	0x20	Comando GPS
3	1	uint8	0x13	Longitud payload (19)
4-22	19	–	–	Payload GPS
23	1	uint8	0xBE	Footer

Cuadro 4: Trama GPS (24 bytes).

3.1. Payload GPS (19 bytes)

Offset	Bytes	Tipo	Campo	Unidad	Notas
0	4	uint32	UnixTime	Segundos	Segundos del día (de GGA)
4	2	uint16	Milliseconds	ms	0-999
6	4	int32	Latitude	deg $\times 10^7$	
10	4	int32	Longitude	deg $\times 10^7$	
14	4	int32	Altitude	mm	Sobre nivel del mar
18	1	uint8	Satellites	Cantidad	

Cuadro 5: Payload GPS (19 bytes).

3.2. Pseudocódigo: construcción de trama GPS en ESP32

Listing 2: ESP32: parsear GGA y construir trama GPS.

```
// Sentencia GGA:
// $GNGGA,134523.500,4042.7688,N,07400.3600,W,1,08,1.0,30.5,M,-34.0,M
// ,,*5E
//          ^^^^^^^^^^  ^^^^^^^^^^  ^  ^^^^^^^^^^^^^^  ^  ^  ^  ^  ^  ^  ^  ^
//          HHMMSS.sss  ddmm.mmmm  dddmm.mmmm  sat  alt(m)

// 1. Parsear hora: "134523.500"
uint32_t hours = 13, minutes = 45, seconds = 23, ms = 500;
uint32_t unix_time = hours * 3600 + minutes * 60 + seconds;
uint16_t milliseconds = ms;

// 2. Parsear latitud: "4042.7688,N"
//  dd = 40, mm.mmmm = 42.7688
//  grados = 40 + 42.7688 / 60 = 40.712813
//  N = positivo, S = negativo
int32_t latitude = (int32_t)(40.712813 * 1e7); // 407128130
```

```

// 3. Parsear longitud: "07400.3600,W"
//   ddd = 074, mm.mmmm = 00.3600
//   grados = 74 + 00.3600 / 60 = 74.006000
//   W = negativo
int32_t longitude = (int32_t)(-74.006000 * 1e7); // -740060000

// 4. Parsear altitud: "30.5" metros
int32_t altitude_mm = (int32_t)(30.5 * 1000); // 30500

// 5. Parsear satelites: "08"
uint8_t satellites = 8;

// 6. Empaquetar payload (19 bytes, little-endian)
uint8_t payload[19];
memcpy(payload + 0, &unix_time, 4); // uint32 LE
memcpy(payload + 4, &milliseconds, 2); // uint16 LE
memcpy(payload + 6, &latitude, 4); // int32 LE
memcpy(payload + 10, &longitude, 4); // int32 LE
memcpy(payload + 14, &altitude_mm, 4); // int32 LE
payload[18] = satellites;

// 7. Construir trama completa (24 bytes)
uint8_t frame[24];
frame[0] = 0xFE; // sync LSB
frame[1] = 0xCA; // sync MSB
frame[2] = 0x20; // COMMAND_GPS_DATA
frame[3] = 0x13; // payload length = 19
memcpy(frame + 4, payload, 19);
frame[23] = 0xBE; // footer

// 8. Enviar por UART al controlador de vuelo
uart_write(frame, 24);

```

4. Resumen

Datos	Dirección	Tamaño	ESP32	ET
Telemetría	CV → ET	52 B	Passthrough	Passthrough
Comandos	ET → CV	5 B	Passthrough	Passthrough
GPS	GPS → CV	24 B	Construir trama	No involucrada

Cuadro 6: CV = Controlador de Vuelo, ET = Estación Terrestre.

5. Estación Terrestre

La estación terrestre es un passthrough bidireccional entre el módulo LoRa y una interfaz de usuario conectada por serial.

- **Telemetría:** recibe los 52 bytes por LoRa y los escribe tal cual por serial para que la interfaz de usuario los lea y decodifique.
- **Comandos:** recibe los 5 bytes por serial desde la interfaz de usuario y los envía tal cual por LoRa.

La estación terrestre no parsea, valida ni modifica ningún paquete. Toda la lógica de decodificación y construcción de comandos está en la interfaz de usuario.